

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس یادگیری تقویتی

تمرین دوم

نام و نام خانوادگی	امیر غرقابی
شماره دانشجویی	۸۱۰۱۰۲۲۱۷
تاریخ ارسال گزارش	۱۴۰۳.۰۸.۲۹

فهرست

۱	پاسخ ۱
۳	پاسخ ۲
۴	پاسخ ۳
۱۰	پاسخ ۴

شکل‌ها

- شکل ۱. صفحه شطرنجی Environment ۴
- شکل ۲. سیاست‌های بهینه برای سناریوی دوم ۵
- شکل ۳. پاسخ بهینه سناریوی سوم به همراه stateهایی که با یکدیگر merge شده اند. ۷
- شکل ۴. نتایج پیاده سازی الگوریتم های Value iteration و Policy iteration در بخش پیاده سازی ۳-الف ۸
- شکل ۵ جهت حرکت agent در هر دو الگوریتم در بخش پیاده سازی ۳-الف ۸
- شکل ۶. نتایج پیاده سازی الگوریتم های Value iteration و Policy iteration در بخش پیاده سازی ۳-ب ۹
- شکل ۷. جهت حرکت agent در هر دو الگوریتم در بخش پیاده سازی ۳-ب ۹
- شکل ۸. نتایج مسئله‌ی finite با الگوریتم Value iteration به طوری که عامل تنها سیاست خرید را دنبال کند ۱۲
- شکل ۹. نتایج مسئله‌ی finite با الگوریتم Policy iteration به طوری که عامل تنها سیاست خرید را دنبال کند ۱۲
- شکل ۱۰. نتایج مسئله‌ی finite با الگوریتم Value iteration به طوری که عامل تنها سیاست فروش را دنبال کند ۱۳
- شکل ۱۱. نتایج مسئله‌ی finite با الگوریتم Policy iteration به طوری که عامل تنها سیاست فروش را دنبال کند ۱۳
- شکل ۱۲. نتایج مسئله‌ی finite با الگوریتم Value iteration به طوری که عامل هم سیاست خرید و هم فروش را دنبال کند. ۱۳
- شکل ۱۳. نتایج مسئله‌ی finite با الگوریتم Policy iteration به طوری که عامل هم سیاست خرید و هم فروش را دنبال کند. ۱۴
- شکل ۱۴. نتایج مسئله‌ی infinite با الگوریتم value iteration با $\lambda = 0$ ۱۴
- شکل ۱۵. نتایج مسئله‌ی infinite با الگوریتم Policy iteration با $\lambda = 0$ ۱۵
- شکل ۱۶. نتایج مسئله‌ی infinite با الگوریتم Value iteration با $\lambda = 0.1$ ۱۶
- شکل ۱۷. نتایج مسئله‌ی infinite با الگوریتم Policy iteration با $\lambda = 0.1$ ۱۶
- شکل ۱۸. نتایج مسئله‌ی infinite با الگوریتم Value iteration با $\lambda = 0.9$ ۱۷
- شکل ۱۹. نتایج مسئله‌ی infinite با الگوریتم Policy iteration با $\lambda = 0.9$ ۱۷

تفاوت بین یادگیری تقویتی و یادگیری نظارتی:

آن ها دو روش مختلف در زیر مجموعه ی یادگیری ماشین هستند که از جهات مختلفی با یکدیگر تفاوت دارند.

۱. نوع داده: در یادگیری نظارتی، برای یادگیری مدل (تابعی که ورودی را به برچسب map می کند) نیاز به دیتاستی است که هر داده برچسب گذاری شده باشد تا مدل یاد گرفته شود. اما در RL، نیاز به دیتاست برچسب گذاری شده نیست. البته RL، متکی بر بازخوردهایی است که از محیط می گیرد. درواقع با هر تصمیم، فیدبکی به عامل داده می شود و عامل با توجه به میزان مثبت بودن این فیدبک، آموزش می بیند که تصمیمش صحیح بوده است یا خیر.

۲. مکانیزم فیدبک: در یادگیری نظارتی، فیدبک مشخص است و به ازای هر نمونه ی آموزشی، معیاری از خطا قابل محاسبه است. اما در RL، این فیدبک ممکن است با تاخیر و یا حتی به صورت sparse به عامل داده شود.

۳. نوع یادگیری: در یادگیری نظارتی، هدف یادگیری و تخمین تابعی است که ورودی را به خروجی map کند اما در RL، هدف یادگیری یک policy است که نحوه انتخاب هر اکشن را در هر state مشخص می کند.

مسئله: پیشبینی قیمت خانه و قیمت گذاری املاک

پیشبینی قیمت خانه:

روش یادگیری نظارتی برای پیشبینی قیمت خانه، نیازمند دیتاستی از انواع خانه ها به همراه برچسب آن ها - یعنی قیمت - است. یعنی دیتاست ورودی الگوریتم های یادگیری نظارتی (مانند رگرسیون خطی) نیاز به متراژ خانه، تعداد اتاق - تعداد پارکینگ و ... به همراه قیمت هر خانه دارد تا بتواند مدلی را یاد بگیرد که با دریافت اطلاعات خانه، قیمت آن را پیشبینی کند.

در مقابل، الگوریتم های RL نیازمند چنین اطلاعاتی نیستند. این روش ها برای هر خانه قیمتی تعیین می کنند و به طور مثال در سایت قرار می دهند. Rewardی که دریافت میکنند ممکن است تعداد کلیک روی تبلیغ آن خانه، بازدید از آن و یا حتی خرید خانه باشد. به ازای هر کدام از این اکشن ها، عامل RL یک میزان reward دریافت میکند. بدین ترتیب عامل سعی میکند تا میزان reward خود را با قیمت های مناسبتری که باعث می شود خریداران روی تبلیغ کلیک کرده

و از طرف دیگر فروشنده نیز راضی به فروش با آن قیمت باشد را انتخاب کند (چون با فروش خانه، میزان reward بسیار بالاتر است). بدین ترتیب قیمت هایی پیشنهاد می دهد که فروشنده و خریدار هر دو راضی باشند. [1]

$$V^*(s) = \max_{a \in A} q_{\pi^*}(s, a)$$

$$= \max_{\pi^*} E [G_t | s_t = s, A_t = a]$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots \quad R_t \leftarrow C R_t \rightarrow$$

$$G'_t = C R_{t+1} + C \gamma R_{t+2} + \dots \Rightarrow G'_t = C G_t$$

$\Rightarrow$

$$V'^*(s) = \max_{\pi^*} E [G'_t | s_t = s, A_t = a]$$

$$= \max_{\pi^*} E [C G_t | s_t = s, A_t = a]$$

$$= \max_{\pi^*} C E [G_t | s_t = s, A_t = a]$$

در روابط بالا فرض گرفته شده است که هر reward در ضریب C ضرب شده است (تبدیل خطی). از آنجایی که عملگر Expected value یک عملگر خطی است، این ضریب از میانگین خارج می شود. و مشاهده می شود که در نهایت $V^*(s) = \max C \times E[.]$ بدست آمده است. (البته می توانستیم تبدیل خطی را بصورت $R' = C \times R + d$ نیز در نظر گرفت که در کلیت روابط تفاوتی ایجاد نمی کرد)

اگر $C > 0$: در این صورت هر اکشنی که reward مثبت تری داشته باشد، همچنان اکشن بهینه بوده و G_t ماکسیمم را ایجاد خواهد کرد بنابراین در سیاست بهینه تغییری ایجاد نخواهد شد. همچنین مقدار d نیز تنها reward ها را شیفت میداد که در تابع max تفاوتی ایجاد نمی کند.

اگر $C=0$: آنگاه در تمامی اکشن ها، مقدار reward برابر با مقدار ثابت d می شد و بدین reward ها روی تصمیمات عامل اثری نخواهند گذاشت. درواقع تمامی اکشن ها میزان $reward=d$ را ایجاد خواهند کرد. بنابراین عامل بصورت رندوم عمل خواهد کرد. البته ممکن است در برخی از state ها، برخی از اکشن ها feasible نباشند. اما در بین اکشن هایی که feasible هستند، عامل بصورت رندوم عمل خواهد کرد.

اگر $C<0$: در این صورت هر اکشنی که reward مثبت تری داشته باشد در سیاست بهینه اثر عکس خواهد گذاشت و در این حالت عامل همواره اکشنی با کمترین reward ممکن را انتخاب خواهد کرد. یعنی اکشنی که در حالت اولیه بهینه است، دیگر بهینه نخواهد بود. در این صورت سیاست بهینه تغییر خواهد کرد. [2]

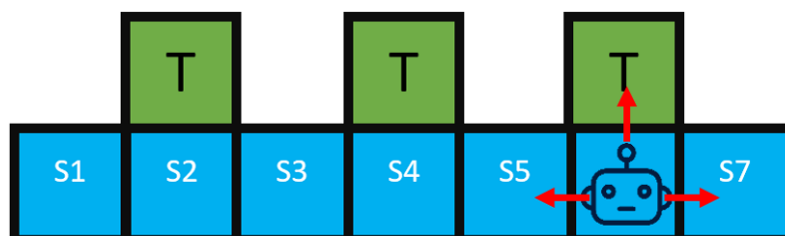
آیا episodic/continuous بودن تسک تاثیری در پاسخ قسمت قبل دارد؟ بطور کلی خیر.

در تسک های episodic که هر iteration در یک تعدادی اکشن (و یا رسیدن به یک terminal state) به پایان می رسد، همواره در حالت $C>0$ ، سیاست بهینه تغییری نکرده و در حالت $C<0$ سیاست بهینه تغییر می کند.

در تسک های continuous نیز که عامل در محیط به صورت بینهایت بار تعامل می کند، شرایط مشابهی برقرار است و در حالت $C>0$ سیاست بهینه تغییری نخواهد داشت. [3]

پاسخ ۳.

در این مسئله، صفحه شطرنجی زیر در نظر گرفته می شود.



شکل ۱. صفحه شطرنجی Environment

سناریو اول:

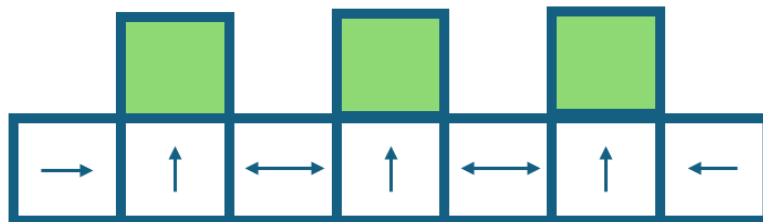
الف.

بله این تسک را می‌توان با یک MDP نمایش داد.

	s_1	s_2	s_3	s_4	s_5	s_6	s_7
<i>up</i>	0	1	0	1	0	1	0
<i>down</i>	0	0	0	0	0	0	0
<i>right</i>	1	1	1	1	1	1	0
<i>left</i>	0	1	1	1	1	1	1

ب.

چند تا سیاست بهینه؟ ۴ تا سیاست بهینه وجود دارد (چون در s_3 و s_5 هر کدام دو سیاست بهینه وجود دارد)



شکل ۲. سیاست‌های بهینه برای سناریوی دوم

سناریو دوم:

الف.

در اینجا ربات به طور قطعی نمیداند که در کدام state قرار دارد. اما می‌تواند برخی از state ها را تفکیک کند. چون اگر تنها به سمت چپ دیوار نباید در O_7 قرار دارد و اگر تنها از طرف پایین امکان حرکت نداشت یعنی در حالت های O_2, O_4 و یا O_6 قرار دارد.

بطور مثال اگر سیگنال $\{up=0, down=0, right=1, left=1\}$ به ربات داده شود، ربات ممکن است در O_5 و یا O_3 باشد. و یا اگر سیگنال $\{up=1, down=0, right=1, left=1\}$ به ربات داده شود، ربات ممکن است در O_2, O_4, O_6 باشد. بنابراین در این حالت این state ها merge خواهند شد.

اگر شروع حرکت ربات از O_1 باشد، در مرحله بعدی ربات به O_2 خواهد رفت.

اگر از O7 شروع کند به O6 خواهد رفت.

اما اگر در یکی از state های O2, O4, O6

در این حالت، مسئله از حالت MDP به Partially Observable MDP تغییر خواهد کرد.

	s_1	$s_{2,4,6}$	$s_{3,5}$	s_7
<i>up</i>	0	1	0	0
<i>down</i>	0	0	0	0
<i>right</i>	1	1	1	0
<i>left</i>	0	1	1	1

ب.

چون در هر ۳ خانه ای که reward داده می شود، reward ها برابر هستند بنابراین سیاست بهینه همچنان همان چیزی خواهد بود که ما می خواهیم.

سناریو سوم:

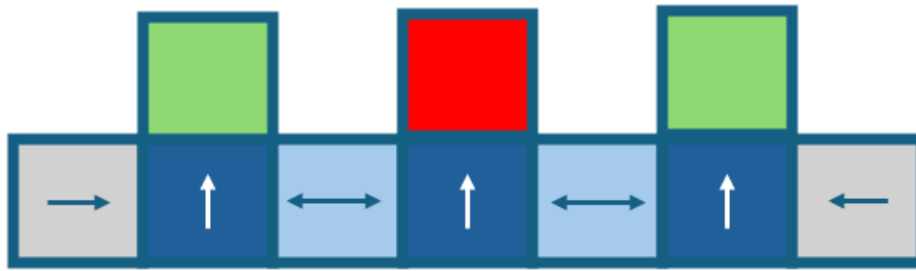
الف.

در اینجا همچنان می توان بصورت MDP مسئله را نشان داد. اما سیاست بهینه ای که به دست خواهد آمد، چیزی که ما می خواهیم نخواهد بود. درواقع اگر ربات در state های 3 و 5 باشد، به احتمال $\frac{2}{3}$ تصمیم بهینه گرفته خواهد شد و به احتمال $\frac{1}{3}$ به $\text{reward} = -1$ خواهد رسید.

یک policy که همواره بهینه عمل خواهد کرد این است که به ازای هر state ی که قرار داریم، آنقدر به سمت راست حرکت کنیم تا سیگنال سیگنال $\{up=0, down=0, right=0, left=1\}$ برسیم (یعنی به S7 state) آنگاه اکشن left و سپس اکشن up انتخاب شود. در این حالت همواره $\text{reward}=1$ دریافت خواهیم کرد.

ب.

پاسخ بهینه مربوط به partially observable MDP خواهد بود.



شکل ۳. پاسخ بهینه سناریوی سوم به همراه stateهایی که با یکدیگر merge شده اند.

در تصویر فوق، state هایی که با یکدیگر ادغام شده اند با رنگ های یکسان مشخص هستند.

	S_1	$S_{2,4,6}$	$S_{3,5}$	S_7
<i>up</i>	0	1	0	0
<i>down</i>	0	0	0	0
<i>right</i>	1	1	1	0
<i>left</i>	0	1	1	1

ماتریس بالا نشان می دهد که از S_1 تنها با اکشن *right* می توان حرکت کرد (به S_{246} state رفت)، از S_{35} نیز می توان با اکشن های *right* و *left* به S_{246} رفت و به همین ترتیب برای سایر stateها.

پیاده سازی سوال ۳:

الف:

در این بخش الگوریتم های *value iteration* و *policy iteration* برای ماتریس *action-state* بخش ۳-ب پیاده سازی شده و نتایج گزارش می شوند. درواقع برخی از state ها ادغام شده و خانه های ترمینال نیز به احتمال $2/3$ ، $reward=1$ داده و به احتمال $1/3$ ، $reward = -1$ خواهد بود. الگوریتم *Value iteration* و *policy iteration* پیاده سازی شده [۴] و نتایج به شرح زیر است.

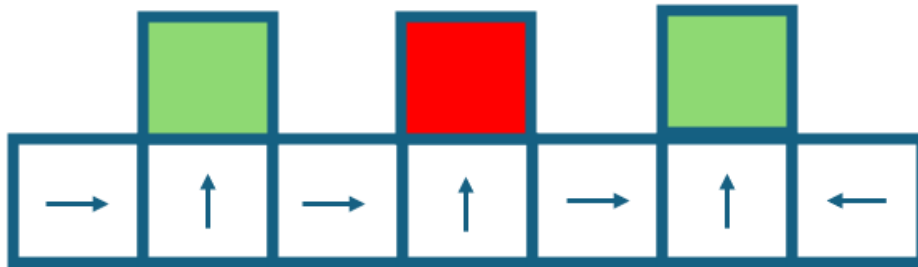
Optimal policy of Value Iteration algorithm _ Question 3.a

```
{'s1': 'right', 's246': 'up', 's35': 'right', 's7': 'left', 'T246': 'Terminal'}
```

Optimal policy of Policy Iteration algorithm _ Question 3.a

```
{'s1': 'right', 's246': 'up', 's35': 'right', 's7': 'left', 'T246': 'Terminal'}
```

شکل ۴. نتایج پیاده سازی الگوریتم های **Value iteration** و **Policy iteration** در بخش پیاده سازی ۳-الف



شکل ۵ جهت حرکت **agent** در هر دو الگوریتم در بخش پیاده سازی ۳-الف

مشاهده می شود که در هر دو الگوریتم، اکشن های بهینه در هر state، دقیقاً اکشن هایی هستند که منجر به رسیدن به state های T246 می شود. اما در این حالت، این terminal state ها تنها با احتمال ۶۶٪ پاداش ۱ به عامل داده و با احتمال ۳۳٪ پاداش ۱- به عامل خواهند داد.

ب:

در این بخش الگوریتم های policy iteration و value iteration برای حالتی اجرا می شود که عامل به طور دقیق از محل قرار گیری خود آگاهی دارد.

در این بخش از متدی به نام self.transition استفاده شده است که با توجه به محل قرارگیری فعلی عامل، اکشن و 's، احتمالی را برای انجام شدن اکشن و همچنین میزان reward به ازای آن اختصاص می دهد. بطور مثال:

Current_state = S2, action = 'right', next_state = S5 → p=0 , reward = 0

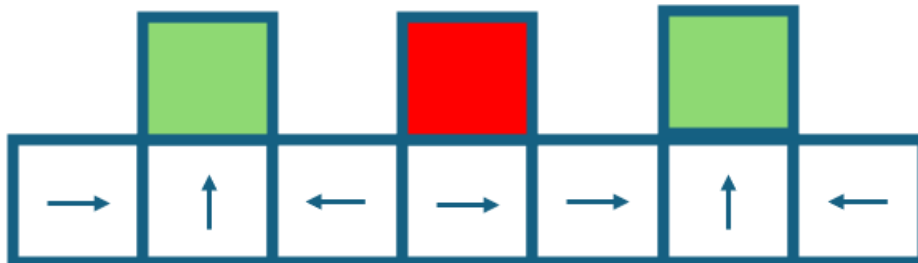
Current_state = S2, action = 'up', next_state = T2 → p=1 , reward = 1

نتایج به شرح زیر است.

```
Optimal policy of Value Iteration algorithm _ Question 3.b
{'s1': 'right', 's2': 'up', 's3': 'left', 's4': 'right', 's5': 'right', 's6': 'up', 's7': 'left', 'T2': 'Terminal', 'T4': 'Terminal', 'T6': 'Terminal'}

Optimal policy of Policy Iteration algorithm _ Question 3.b
{'s1': 'right', 's2': 'up', 's3': 'left', 's4': 'right', 's5': 'right', 's6': 'up', 's7': 'left', 'T2': 'Terminal', 'T4': 'Terminal', 'T6': 'Terminal'}
```

شکل ۶. نتایج پیاده سازی الگوریتم های Value iteration و Policy iteration در بخش پیاده سازی ۳-ب



شکل ۷. جهت حرکت agent در هر دو الگوریتم در بخش پیاده سازی ۳-ب

مشاهده می شود که در هر دو الگوریتم، اکشن های بهینه در هر state، دقیقاً اکشن هایی هستند که منجر به رسیدن به state های T2 و T6 می شوند.

تفاوت بخش های الف و ب در ماتریس احتمال (state action prob matrix) است

زمان سپری شده در اجرای هر الگوریتم در دو بخش الف و ب به صورت جدول زیر قابل نمایش است.

جدول ۱. مقایسه زمان اجرای الگوریتم های Value iteration و Policy iteration در سوال ۳

الگوریتم	زمان اجرا الف	زمان اجرا ب
Value iteration	0.00013	0.0004
Policy iteration	0.00006	0.0002

در هر دو بخش الف و ب، هر دو الگوریتم با سرعت بسیار خوبی عمل کرده اند (البته این اتفاق منتظره بود زیرا تعداد state ها و action ها بسیار کم بوده است) اما الگوریتم Policy iteration به نسبت، سریعتر عمل کرده است.

پاسخ ۴.

مسئله ی خرید و فروش توسط یک عامل.

الف.

بله. مقدار H می تواند ۱۵ باشد. در این صورت عامل می تواند ۳ بار اکشن فروش، سپس ۱ بار خرید، ۱ بار فروش و سپس ۱۰ بار خرید انجام دهد. در این صورت هم عمل خرید را انجام داده، هم فروش را و هم به $s=10$ و دریافت $\text{reward} = 100$ رسیده است. اگر عمل خرید را انجام نמידاد، ۱ بار کمتر می توانست بفروشد و در ۱۴ action به مقدار $s=10$ می رسید. اما در حالتی که گفته شده، ۱ پاداش بیشتر دریافت می کند. بنابراین می توان از حالت $s=3$ شروع کرده و Policy ای را اجرا کرد که با استفاده از اکشن خرید و فروش (با هم) می تواند به بیشینه پاداش دست یابد.

ب.

برای رسیدن به موجودی کاملاً پر، حداقل ۷ مورد اکشن خرید باید اجرا شود یعنی $H=7$. و در این حین نیز سیاست بهینه نیازی به فروش (و دریافت پاداش ۱ ندارد) زیرا تنها با استفاده از اکشن خرید، به پاداش 100 خواهد رسید. اگر $H=9$ باشد، نه تنها می توان به موجودی $s=10$ رسید بلکه می توان یک بار هم از اکشن فروش و سپس خرید استفاده کرد.

بنابراین اگر $H \geq 7$ باشد، سیاست بهینه عامل را به موجودی کاملاً پر رسانده و $\text{reward} = 100$ را دریافت کرده و اپیزود تمام می شود.

اما ممکن است H انقدر زیاد باشد که تعداد خرید و فروش ها زیاد باشد و عامل را به $\text{reward} > 100$ برساند. در این حالت هم می توان سیاستی را یافت که در اکشن H ام (یا $H-1$ ام) به $s=10$ رسیده و $\text{reward} += 100$ شود.

البته اگر $H = \text{inf}$ در نظر گرفته شود، بهتر است هیچگاه به $s=10$ نرسیده و همواره عملیات خرید و فروش را در $S < 9$ انجام دهد.

ج.

در حالت $\lambda = 0$ تنها آخرین اکشن، مقدار value function را تغییر می دهد. اگر مسئله افق نامحدود در نظر گرفته شود، الگوریتم تنها زمانی متوقف می شود که به یک terminal state برسد که در این مسئله $s=10$ به عنوان terminal state در نظر گرفته شده است. چون در این حالت، مسئله بیشینه کردن instant

reward است، سیاست بهینه این است که عامل ۳ بار فروش انجام دهد تا به $s=0$ برسد. سپس یکی در میان خرید و فروش انجام دهد و این کار را بینهایت بار تکرار کند.

اگر از $s=9$ شروع کند، سیاست بهینه این است که یک خرید انجام دهد و بدین ترتیب MDP به پایان برسد.

د.

بله وجود دارد. در ابتدا فرض میکنیم سیاست بهینه با $\lambda < 1$ این است که عامل به $s=10$ برسد. برای اینکه بیشترین reward دریافت شود، لازم است که در سریعترین زمان ممکن به این state برسد. بنابراین باید ۷ خرید پشت سر هم انجام دهد که میزان return برابر است با:

$$G_{s=3} = 0 + \lambda \times 0 + \lambda^2 \times 0 + \dots + \lambda^5 \times 0 + \lambda^6 \times 100$$

اگر سیاست بهینه اینگونه باشد که هرگز موجودی را کامل پر نکند میزان return در حالتی محاسبه می شود که در ابتدا ۳ فروش انجام شود و سپس یکی در میان خرید و فروش انجام شود. بنابراین میزان return برابر خواهد بود با:

$$\begin{aligned} G'_{s=3} &= 1 + \lambda \times 1 + \lambda^2 \times 1 + \lambda^3 \times 0 + \lambda^4 \times 1 + \lambda^5 \times 0 + \lambda^6 \times 1 + \dots \\ &= 1 + \lambda \times 1 + \lambda^2 (1 + \lambda^2 + \lambda^4 + \dots) \\ &= 1 + \lambda \times 1 + \lambda^2 \left(\frac{1}{1 - \lambda^2} \right) \end{aligned}$$

بنابراین اگر سیاست بهینه اینگونه باشد که عامل هرگز به $s=10$ نرسد، باید $G' > G$ باشد. با حل این نامعادله، مقادیری برای $\lambda^* \in \lambda \in [0, 1)$ بدست می آید که با در نظر گرفتن آن، سیاست بهینه به صورتی خواهد بود که عامل هرگز به $s=10$ نرسد.

پیاده سازی:

در این بخش مسئله به دو صورت finite horizon و infinite horizon شبیه سازی می شود.

اگر horizon در مسئله لحاظ شود، آن گاه علاوه بر state فعلی عامل، مرحله ای که در horizon قرار دارد هم اهمیت پیدا کرده و در سیاست بهینه تاثیر گذار خواهد بود. بنابراین برای هر state و مرحله ی horizon، یک زوج مرتب در نظر گرفته می شود. (درواقع تعداد state ها افزایش خواهند یافت). الگوریتم های policy iteration و value iteration در هر دو حالت finite و infinite پیاده سازی شده [۵] و نتایج در ادامه ارائه خواهند شد.

پیاده سازی مسئله به صورت **finite horizon**:

الف. مقدار H ی که عامل با توجه به آن تنها سیاست خرید را دنبال کند.

```
Optima Value Iteration policy for gamma: 1 and Horizon: 7
step 7: State = 3, action = buy
step 6: State = 4, action = buy
step 5: State = 5, action = buy
step 4: State = 6, action = buy
step 3: State = 7, action = buy
step 2: State = 8, action = buy
step 1: State = 9, action = buy
Total Reward: 100
```

شکل ۸. نتایج مسئله **finite** با الگوریتم **Value iteration** به طوری که عامل تنها سیاست خرید را دنبال کند

```
Optima Policy Iteration policy for gamma: 1 and Horizon: 7
step 7: State = 3, action = buy
step 6: State = 4, action = buy
step 5: State = 5, action = buy
step 4: State = 6, action = buy
step 3: State = 7, action = buy
step 2: State = 8, action = buy
step 1: State = 9, action = buy
Total Reward: 100
```

شکل ۹. نتایج مسئله **finite** با الگوریتم **Policy iteration** به طوری که عامل تنها سیاست خرید را دنبال کند

مشاهده می شود که اگر $H=7$ باشد، سیاست بهینه برای عامل این است که به $s=10$ رسیده و $\text{reward}=100$ را دریافت کند.

ب. مقدار H ی که عامل با توجه به آن تنها سیاست فروش را دنبال کند.

```
Optima Value Iteration policy in Finite horizon for H: 3
step 3: State = 3, action = sell
step 2: State = 2, action = sell
step 1: State = 1, action = sell
Total Reward: 3
```

شکل ۱۰. نتایج مسئله‌ی **finite** با الگوریتم **Value iteration** به طوری که عامل تنها سیاست فروش را دنبال کند

```
Optima Policy Iteration policy for gamma: 1 and Horizon: 3
step 3: State = 3, action = sell
step 2: State = 2, action = sell
step 1: State = 1, action = sell
Total Reward: 3
```

شکل ۱۱. نتایج مسئله‌ی **finite** با الگوریتم **Policy iteration** به طوری که عامل تنها سیاست فروش را دنبال کند

مشاهده می شود که با در نظر گرفتن $\lambda=1$ و $H=3$ سیاست بهینه تنها شامل اکشن فروش خواهد شد. زیرا با شروع از $state=3$ نمی تواند به $s=10$ رسیده و $reward=100$ را دریافت کند.

ج. سیاست بهینه هم خرید و هم فروش را دنبال کند.

```
Optima Value Iteration policy in Finite horizon for H: 9
step 9: State = 3, action = sell
step 8: State = 2, action = buy
step 7: State = 3, action = buy
step 6: State = 4, action = buy
step 5: State = 5, action = buy
step 4: State = 6, action = buy
step 3: State = 7, action = buy
step 2: State = 8, action = buy
step 1: State = 9, action = buy
Total Reward: 101
```

شکل ۱۲. نتایج مسئله‌ی **finite** با الگوریتم **Value iteration** به طوری که عامل هم سیاست خرید و هم فروش را دنبال کند.


```

Optima Policy Iteration policy for gamma: 1 and Horizon: 9
step 9: State = 3, action = sell
step 8: State = 2, action = buy
step 7: State = 3, action = buy
step 6: State = 4, action = buy
step 5: State = 5, action = buy
step 4: State = 6, action = buy
step 3: State = 7, action = buy
step 2: State = 8, action = buy
step 1: State = 9, action = buy
Total Reward: 101

```

شکل ۱۳. نتایج مسئله‌ی **finite** با الگوریتم **Policy iteration** به طوری که عامل هم سیاست خرید و هم فروش را دنبال کند.

مشاهده می شود که با در نظر گرفتن $\lambda = 1$ اگر $H = 9$ سیاست بهینه شامل هر دو اکشن خرید و فروش خواهد شد.

پیاده سازی مسئله به صورت **infinite horizon**:

ب. به ازای $\lambda = 0$

```

Optimal Value Iteration policy for gamma: 0
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Buy

```

شکل ۱۴. نتایج مسئله‌ی **inifinte** با الگوریتم **value iteration** با $\lambda = 0$

```
Optima Policy Iteration policy for gamma: 0
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Buy
```

شکل ۱۵. نتایج مسئله‌ی **inifinte** با الگوریتم **Policy iteration** با $\lambda = 0$

در حالت $\lambda = 0$ ، تنها مقدار instant reward اهمیت خواهد داشت بنابراین بهترین اکشن برای عامل این است که سهام های خود را بفروشد. و هنگامی که سهام هایش تمام شد، سهام خریداری کرده و سپس دوباره آن را بفروشد.

همانطور که در بخش ج نیز به آن اشاره شد، اگر عامل در $s=9$ قرار داشته باشد، در صورت خرید سهام، instant reward آن برابر با 100 خواهد شد بنابراین یک بار خرید انجام می دهد تا پاداش لحظه ای خود را ماکسیمم کند. این نتیجه در شکل های فوق نیز تصدیق شده است.

ج. به ازای $\lambda = 0.1$

```
Optimal Value Iteration policy for gamma: 0.1
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Buy
Buy
```

شکل ۱۶. نتایج مسئله‌ی **inifinte** با الگوریتم **Value iteration** با $\lambda = 0.1$

```
Optima Policy Iteration policy for gamma: 0.1
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Sell
Buy
Buy
```

شکل ۱۷. نتایج مسئله‌ی **inifinte** با الگوریتم **Policy iteration** با $\lambda = 0.1$

به ازای $\lambda = 0.9$

```
Optimal Value Iteration policy for gamma: 0.9
Sell
Buy
Buy
Buy
Buy
Buy
Buy
Buy
Buy
Buy
Buy
```

شکل ۱۸. نتایج مسئله‌ی **inifinte** با الگوریتم **Value iteration** با $\lambda = 0.9$

```
Optimal Policy Iteration policy for gamma: 0.9
Sell
Buy
Buy
Buy
Buy
Buy
Buy
Buy
Buy
Buy
Buy
```

شکل ۱۹. نتایج مسئله‌ی **inifinte** با الگوریتم **Policy iteration** با $\lambda = 0.9$

مشاهده می شود که اگر $\lambda = 0.9$ در نظر گرفته شود، reward ها نسبت به حالت $\lambda = 0.1$ با سرعت کمتری فراموش می شوند (یعنی دیرتر به 0 میل می کنند). بنابراین سیاست بهینه اینگونه عمل می کند که عامل به $s=10$ رسیده و $\text{reward}=100$ را دریافت کند.

در مقابل، هنگامی که $\lambda = 0.1$ در نظر گرفته شود، اگر قرار باشد عامل به $s=10$ برسد، زمان زیادی لازم است (۷ مرحله) به همین دلیل ضریب $\lambda^7 = 0.1^7$ بسیار کوچک شده و تقریباً $\text{reward}=100$ را از بین می برد. اما با $\lambda = 0.1$ ، سیاست بهینه ی عامل اینگونه خواهد بود که ۲ بار خرید انجام دهد و به $s=10$ برسد چون $G = 0 + \lambda \times 100 = 10$ و ارزش بیشتری از فروش ۸ سهام و دریافت $G=8$ خواهد داشت.

[1] GPT 4-o, Prompt: “give me two analogous problems where one can be addressed with Supervised Learning (SL) and the other requires Reinforcement Learning (RL)”

[2] Inspired by GPT 4-o, Prompt: “in an mdp problem, if the reward function changes under a linear transformation, does the optimal policy change” and “what if $c, 0$ ”

[3] GPT 4-o, Prompt: “is the task being continuing or episodic change the results you just provided for the C?”

[4] By the help of ChatGPT

[5] By the help of ChatGPT